

COGNIZANT DIGITAL NURTURE 5.0

PL/SQL EXERCISE 3 – STORED PROCEDURES

SCENARIO 1: PROCESS MONTHLY INTEREST

Problem Statement:

The bank needs to process monthly interest for all savings accounts by applying an interest rate of 1% to the current balance.

Procedure Name:

ProcessMonthlyInterest

PL/SQL Code:

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
AS
BEGIN
UPDATE Accounts
SET Balance = Balance + (Balance * 0.01)
WHERE AccountType = 'Savings';

COMMIT;

DBMS_OUTPUT.PUT_LINE('Monthly interest applied successfully.');
```

END;

/

Execution:

```
BEGIN
ProcessMonthlyInterest;
END;
```

/

Output Screenshot:

The screenshot shows the SQL Developer interface. The top toolbar includes icons for running, saving, and other database operations. The 'Query Builder' tab is active, displaying the following PL/SQL code:

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
AS
BEGIN
    UPDATE Accounts
    SET Balance = Balance + (Balance * 0.01)
    WHERE AccountType = 'Savings';
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Monthly interest applied successfully.');
```

The code is followed by a call to the procedure and a slash to end the block. Below the code editor, the 'Script Output' and 'Query Result' tabs are visible. The 'Script Output' tab shows the following messages:

```
Task completed in 0.046 seconds

Procedure PROCESSMONTHLYINTEREST compiled

Monthly interest applied successfully.

PL/SQL procedure successfully completed.
```

Verification Query:

SELECT * FROM Accounts;

Verification Screenshot:

The screenshot shows the SQL Developer interface with the 'Query Result' tab active. The query 'SELECT * FROM Accounts;' is entered in the SQL editor. The results are displayed in a table with 4 columns: ACCOUNTID, CUSTOMERNAME, ACCOUNTTYPE, and BALANCE. The table contains 3 rows of data.

	ACCOUNTID	CUSTOMERNAME	ACCOUNTTYPE	BALANCE
1	101	Ravi	Savings	10100
2	102	Priya	Savings	15150
3	103	Kumar	Current	20000

SCENARIO 2: UPDATE EMPLOYEE BONUS

Problem Statement:

The bank wants to implement a bonus scheme for employees based on their performance.

Procedure Name:

UpdateEmployeeBonus

PL/SQL Code:

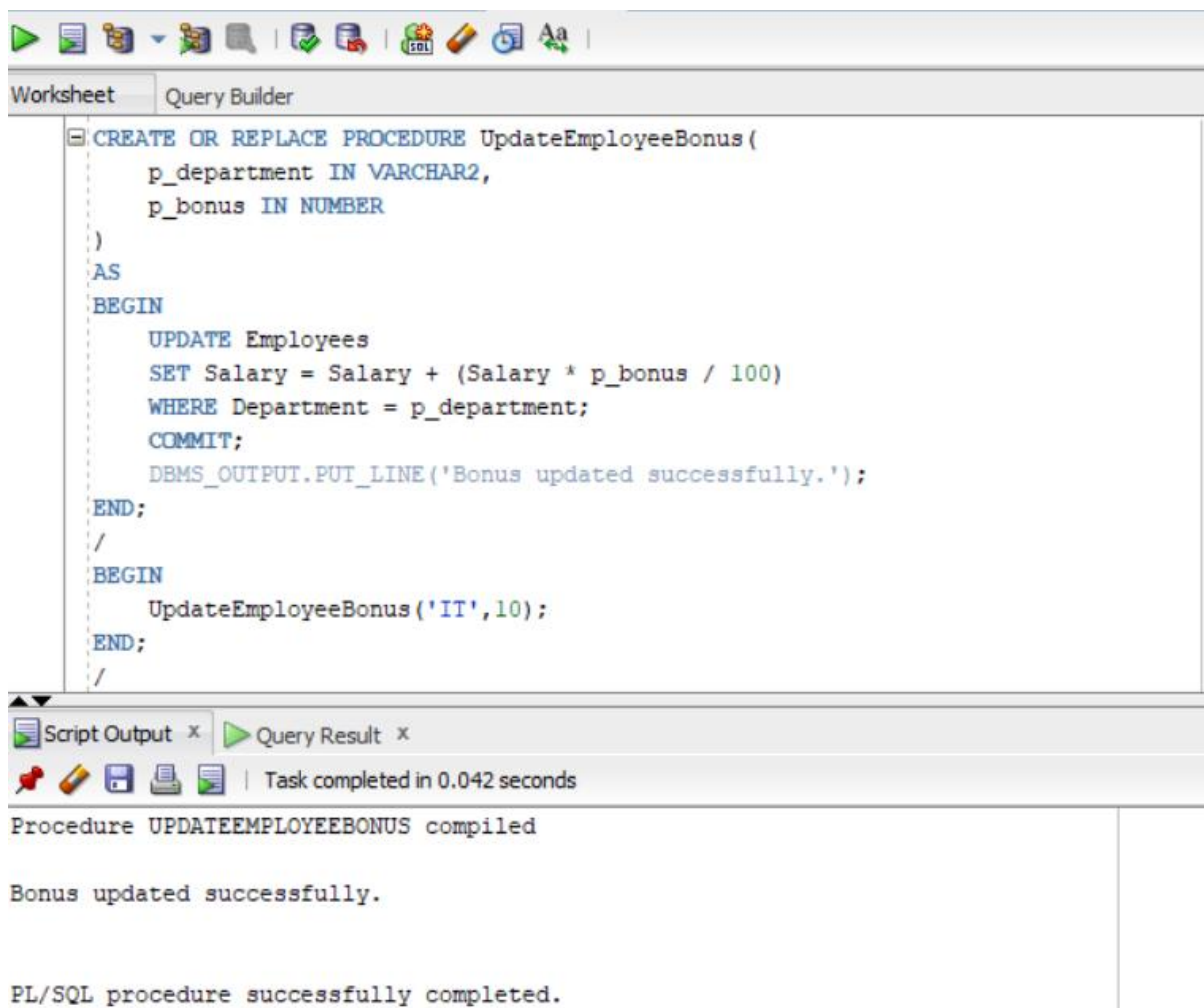
```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(  
  p_department IN VARCHAR2,  
  p_bonus IN NUMBER  
)  
AS  
BEGIN  
  UPDATE Employees  
  SET Salary = Salary + (Salary * p_bonus / 100)  
  WHERE Department = p_department;  
  
  COMMIT;  
  
  DBMS_OUTPUT.PUT_LINE('Bonus updated successfully.');
```

END;
/

Execution:

```
BEGIN  
UpdateEmployeeBonus('IT',10);  
END;  
/
```

Output Screenshot:



The screenshot displays the SQL Developer interface. The top toolbar includes icons for running, saving, and other database operations. The 'Query Builder' tab is active, showing a PL/SQL script. The script defines a procedure 'UpdateEmployeeBonus' that takes a department name and a bonus percentage as input, updates the salary of employees in that department by the specified percentage, and outputs a success message. Below the script, the 'Script Output' and 'Query Result' tabs are visible. The 'Script Output' tab shows the execution results: 'Procedure UPDATEEMPLOYEEBONUS compiled', 'Bonus updated successfully.', and 'PL/SQL procedure successfully completed.'

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(  
    p_department IN VARCHAR2,  
    p_bonus IN NUMBER  
)  
AS  
BEGIN  
    UPDATE Employees  
    SET Salary = Salary + (Salary * p_bonus / 100)  
    WHERE Department = p_department;  
    COMMIT;  
    DBMS_OUTPUT.PUT_LINE('Bonus updated successfully.');
```

END;
/
BEGIN
 UpdateEmployeeBonus('IT',10);
END;
/

Script Output x Query Result x

Task completed in 0.042 seconds

Procedure UPDATEEMPLOYEEBONUS compiled

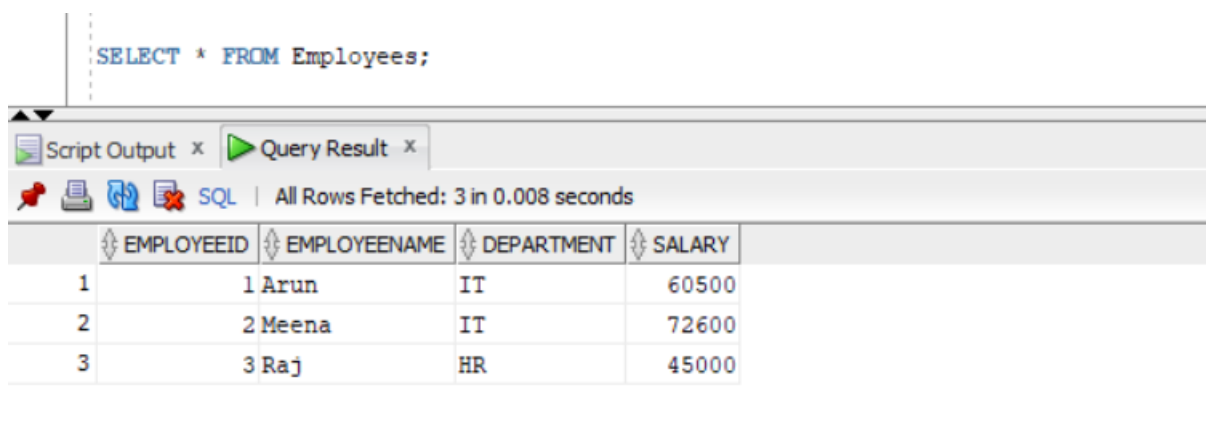
Bonus updated successfully.

PL/SQL procedure successfully completed.

Verification Query:

SELECT * FROM Employees;

Verification Screenshot:



The screenshot shows the SQL Developer interface with the 'Query Result' tab active. It displays the results of the 'SELECT * FROM Employees;' query. The results are shown in a table with four columns: EMPLOYEEID, EMPLOYEENAME, DEPARTMENT, and SALARY. There are three rows of data.

	EMPLOYEEID	EMPLOYEENAME	DEPARTMENT	SALARY
1	1	Arun	IT	60500
2	2	Meena	IT	72600
3	3	Raj	HR	45000

SCENARIO 3: TRANSFER FUNDS

Problem Statement:

Customers should be able to transfer funds between their accounts.

Procedure Name:

TransferFunds

PL/SQL Code:

```
CREATE OR REPLACE PROCEDURE TransferFunds(
p_fromAccount IN NUMBER,
p_toAccount IN NUMBER,
p_amount IN NUMBER
)
AS
v_balance NUMBER;
BEGIN
SELECT Balance
INTO v_balance
FROM Accounts
WHERE AccountID = p_fromAccount;

IF v_balance >= p_amount THEN

    UPDATE Accounts

    SET Balance = Balance - p_amount

    WHERE AccountID = p_fromAccount;

    UPDATE Accounts

    SET Balance = Balance + p_amount

    WHERE AccountID = p_toAccount;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Transfer Successful');

ELSE

    DBMS_OUTPUT.PUT_LINE('Insufficient Balance');

END IF;

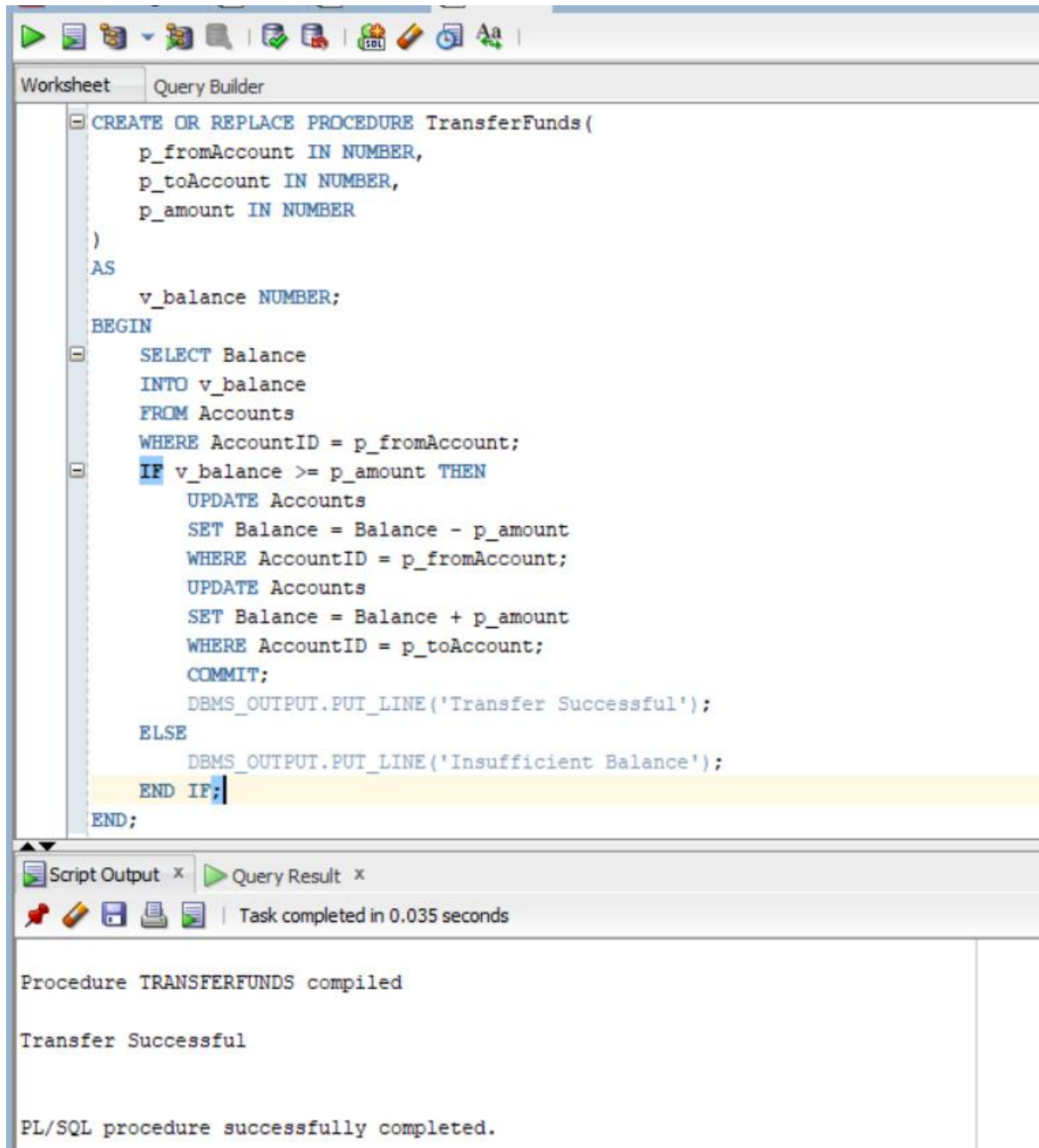
END;

/
```

Execution:

```
BEGIN  
TransferFunds(101,102,2000);  
END;  
/
```

Output Screenshot:



The screenshot displays the Oracle SQL Developer interface. The top toolbar includes icons for running queries, saving, and other database functions. The main window is titled 'Query Builder' and contains the following PL/SQL code:

```
CREATE OR REPLACE PROCEDURE TransferFunds(  
    p_fromAccount IN NUMBER,  
    p_toAccount IN NUMBER,  
    p_amount IN NUMBER  
)  
AS  
    v_balance NUMBER;  
BEGIN  
    SELECT Balance  
    INTO v_balance  
    FROM Accounts  
    WHERE AccountID = p_fromAccount;  
    IF v_balance >= p_amount THEN  
        UPDATE Accounts  
        SET Balance = Balance - p_amount  
        WHERE AccountID = p_fromAccount;  
        UPDATE Accounts  
        SET Balance = Balance + p_amount  
        WHERE AccountID = p_toAccount;  
        COMMIT;  
        DBMS_OUTPUT.PUT_LINE('Transfer Successful');  
    ELSE  
        DBMS_OUTPUT.PUT_LINE('Insufficient Balance');  
    END IF;  
END;
```

Below the code editor, the 'Script Output' and 'Query Result' tabs are visible. The 'Script Output' tab shows the following messages:

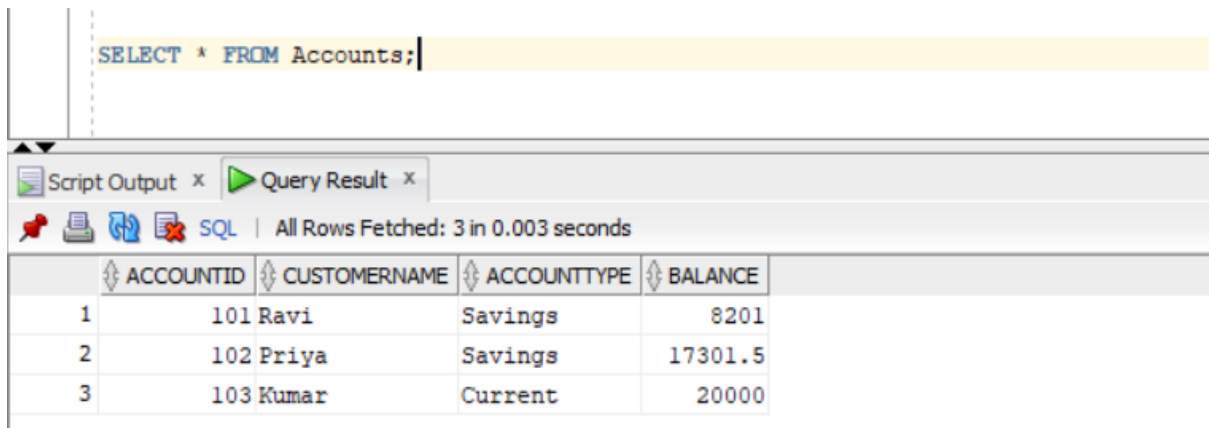
```
Procedure TRANSFERFUNDS compiled  
  
Transfer Successful  
  
PL/SQL procedure successfully completed.
```

The 'Query Result' tab is currently empty. A status bar at the bottom indicates 'Task completed in 0.035 seconds'.

Verification Query:

```
SELECT * FROM Accounts;
```

Verification Screenshot:

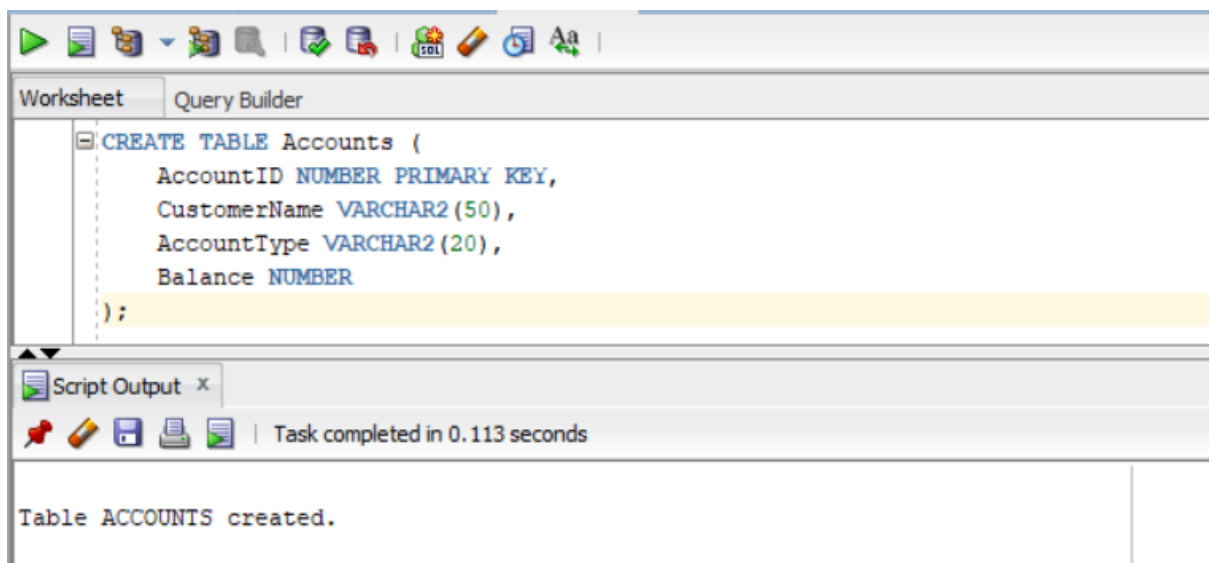


The screenshot shows a database query result. The query is `SELECT * FROM Accounts;`. The result is displayed in a table with 4 columns: ACCOUNTID, CUSTOMERNAME, ACCOUNTTYPE, and BALANCE. There are 3 rows of data.

	ACCOUNTID	CUSTOMERNAME	ACCOUNTTYPE	BALANCE
1	101	Ravi	Savings	8201
2	102	Priya	Savings	17301.5
3	103	Kumar	Current	20000

DATABASE SETUP

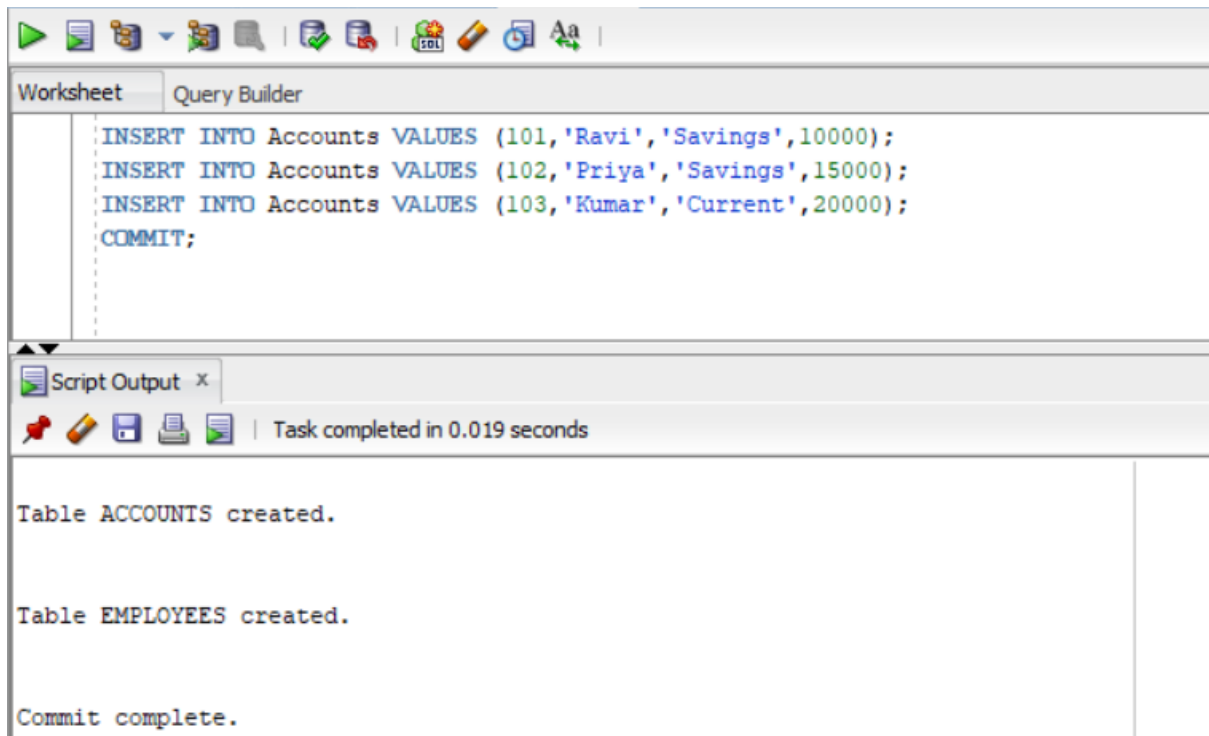
ACCOUNTS TABLE CREATION:



The screenshot shows a database query builder interface. The query is `CREATE TABLE Accounts ( AccountID NUMBER PRIMARY KEY, CustomerName VARCHAR2(50), AccountType VARCHAR2(20), Balance NUMBER );`. The result is displayed in a table with 1 row and 1 column: Table ACCOUNTS created.

Table ACCOUNTS created.

INSERTING ROWS IN ACCOUNTS TABLE :



The screenshot shows the SQL Developer interface. The top toolbar includes icons for running, saving, and editing. Below the toolbar, there are two tabs: 'Worksheet' and 'Query Builder'. The 'Worksheet' tab is active, displaying a SQL script with three INSERT statements and a COMMIT statement. The script is as follows:

```
INSERT INTO Accounts VALUES (101, 'Ravi', 'Savings', 10000);  
INSERT INTO Accounts VALUES (102, 'Priya', 'Savings', 15000);  
INSERT INTO Accounts VALUES (103, 'Kumar', 'Current', 20000);  
COMMIT;
```

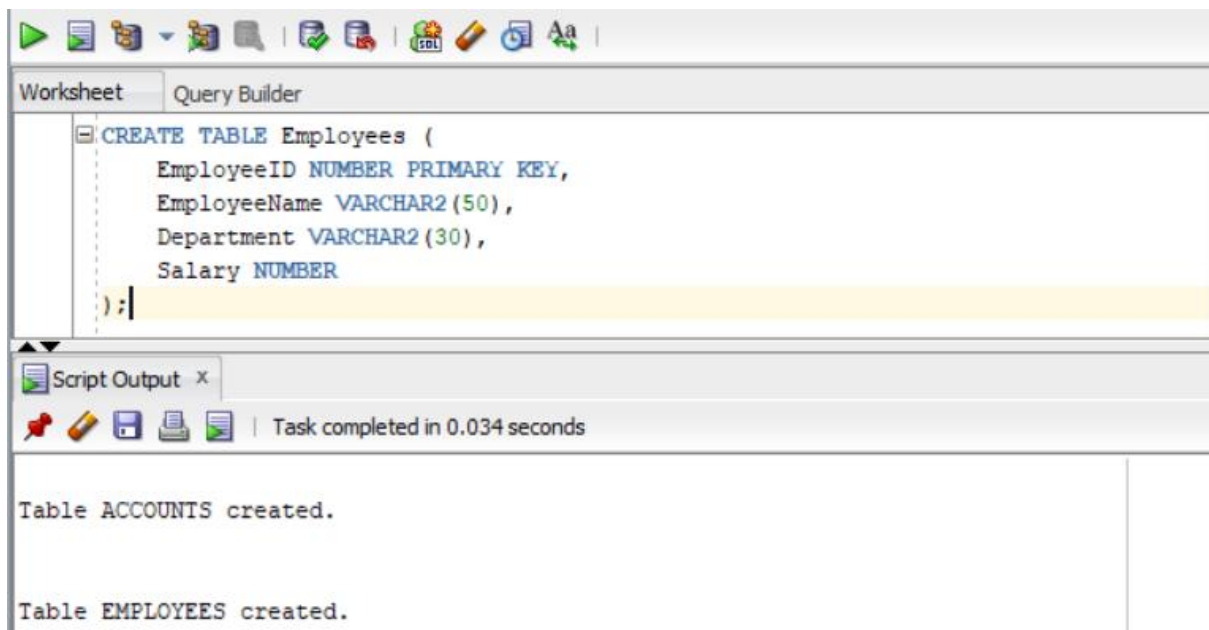
Below the script editor, there is a 'Script Output' window. It shows the results of the script execution, indicating that the tables were created and the commit was successful.

Table ACCOUNTS created.

Table EMPLOYEES created.

Commit complete.

EMPLOYEE TABLE CREATION:



The screenshot shows the SQL Developer interface. The top toolbar includes icons for running, saving, and editing. Below the toolbar, there are two tabs: 'Worksheet' and 'Query Builder'. The 'Worksheet' tab is active, displaying a SQL script for creating the EMPLOYEES table. The script is as follows:

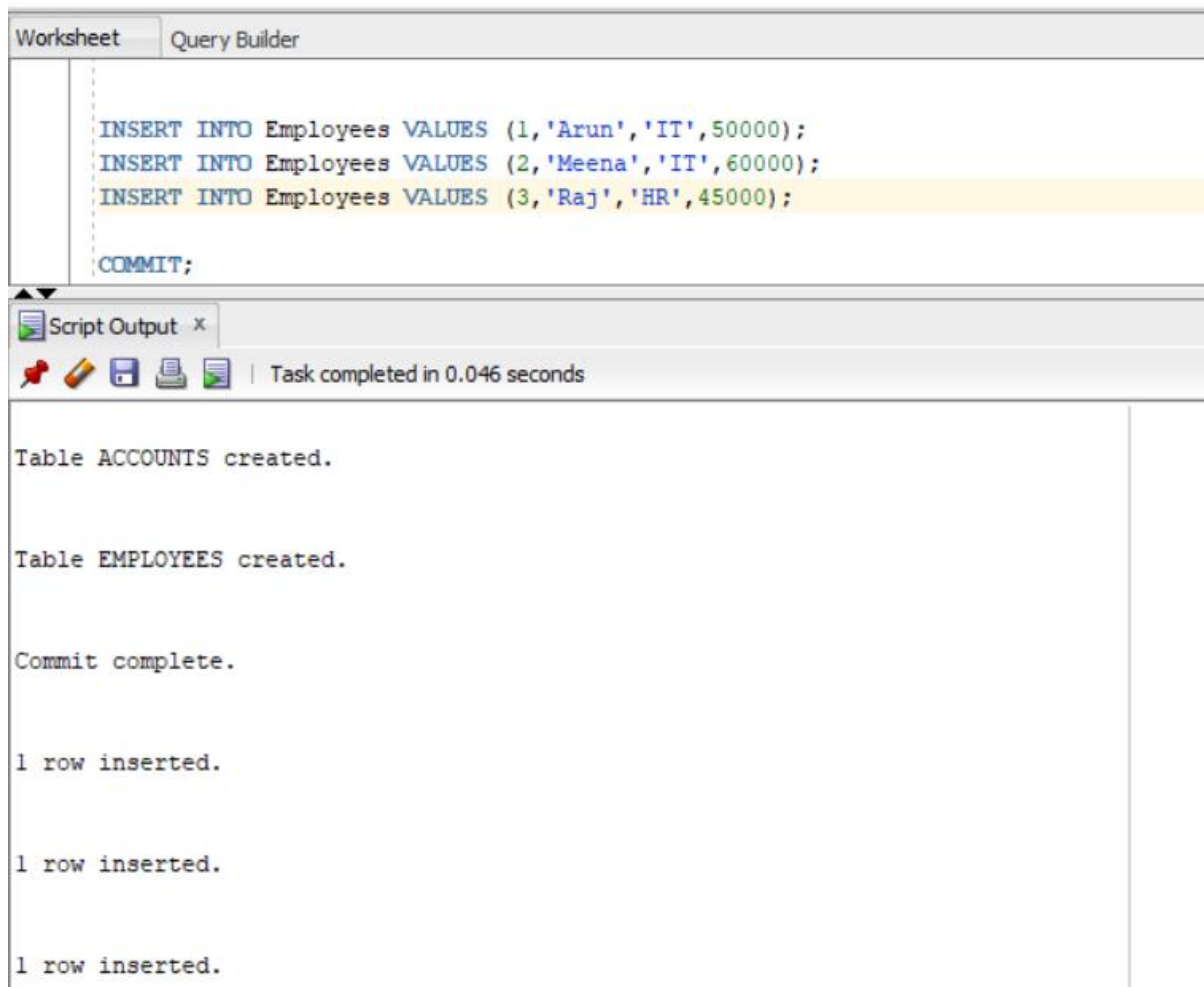
```
CREATE TABLE Employees (  
    EmployeeID NUMBER PRIMARY KEY,  
    EmployeeName VARCHAR2(50),  
    Department VARCHAR2(30),  
    Salary NUMBER  
);
```

Below the script editor, there is a 'Script Output' window. It shows the results of the script execution, indicating that the tables were created.

Table ACCOUNTS created.

Table EMPLOYEES created.

INSERTING ROWS IN EMPLOYEES TABLE:



The screenshot displays the SQL Developer interface. The top pane, titled 'Query Builder', contains the following SQL code:

```
INSERT INTO Employees VALUES (1, 'Arun', 'IT', 50000);  
INSERT INTO Employees VALUES (2, 'Meena', 'IT', 60000);  
INSERT INTO Employees VALUES (3, 'Raj', 'HR', 45000);  
  
COMMIT;
```

The bottom pane, titled 'Script Output', shows the execution results:

```
Table ACCOUNTS created.  
  
Table EMPLOYEES created.  
  
Commit complete.  
  
1 row inserted.  
  
1 row inserted.  
  
1 row inserted.
```

RESULT:

Thus, the PL/SQL Stored Procedures for processing monthly interest, updating employee bonuses, and transferring funds were successfully implemented and executed.